

N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 17

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing

Task 1: Student Academic Data Cleaning

AI Prompt Used

Generate a Python script using pandas to create and clean a student dataset. Include handling missing values, duplicates, standardization, and encoding.

Explanation

In this task, a student dataset is created and cleaned using preprocessing techniques. Missing values in marks, attendance, and department are handled using statistical methods such as mean, median, and mode. Duplicate records are removed based on student ID to ensure data integrity. Text fields such as department names are standardized to maintain consistency. Finally, categorical variables like gender and department are encoded into numerical form for analysis.

Code Screenshot

```
print("Label encoding mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
```

```

0      1  Alice  Female  85.000000    92.0      0      True  False  False
1      2   Bob   Male  78.000000    88.0      1      True  False  False
2      3  Carol  Female  90.000000    88.0      0     False   True  False
3      4  David   Male  82.666667    75.0      1     False  False   True
4      5   Eva  Unknown  82.000000    80.0      2      True  False  False
5      6  Frank   Male  91.000000    95.0      1     False   True  False
6      7  Grace  Female  69.000000    70.0      0     False  False   True
7      8  Henry   Male  74.000000    88.0      1      True  False  False
8      9   Ivy   Female  88.000000    90.0      0      True  False  False
9     10  James   Male  56.000000    88.0      1     False   True  False

DataFrame info:
<class 'pandas.core.frame.DataFrame'>
Index: 20 entries, 0 to 20
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   student_id  20 non-null      int64
1   name        20 non-null      object
2   gender       20 non-null      object
3   marks       20 non-null      float64
4   attendance  20 non-null      float64
5   gender_encoded  20 non-null      int64
6   dept_cse    20 non-null      bool
7   dept_ece    20 non-null      bool
8   dept_me     20 non-null      bool
dtypes: bool(3), float64(2), int64(2), object(2)
memory usage: 1.2+ KB
None

Value counts for gender encoding:
gender
Female    9
Male      9
Unknown   2
Name: count, dtype: int64
8   dept_me     20 non-null      bool
dtypes: bool(3), float64(2), int64(2), object(2)

```

Task 2: Financial Transaction Data Preprocessing

AI Prompt Used

Generate Python code to preprocess transaction data including datetime conversion, feature extraction, normalization, and outlier handling.

Explanation

In this task, financial transaction data is preprocessed to make it suitable for analysis. The transaction date is converted into datetime format to enable time-based analysis. Additional features such as transaction month and year are extracted. The transaction amount is normalized to bring all values into a common scale. Outliers are identified and handled to reduce their impact on analysis.

Code Screenshot

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.preprocessing import MinMaxScaler
4
5  # Step 1: Create sample CSV data
6  np.random.seed(42)
7  n = 35
8  base_date = pd.to_datetime("2023-01-01")
9  data = []
10 for i in range(1, n + 1):
11     tx_id = 1000 + i
12     tx_date = (base_date + pd.to_timedelta(np.random.randint(0, 365), unit="d")).strftime("%Y-%m-%d")
13     amount = np.random.normal(200, 80) # normal cluster
14     if np.random.rand() < 0.1:
15         amount = np.random.choice([1500, 3000, 5000, -200]) # extreme outlier values
16     customer_id = np.random.randint(1, 15)
17     data.append({
18         "transaction_id": tx_id,
19         "transaction_date": tx_date,
20         "transaction_amount": round(amount, 2),
21         "customer_id": customer_id,
22     })
23
24 df = pd.DataFrame(data)
25 df.to_csv("transactions.csv", index=False)
26 print("Created transactions.csv\n")
27
28 # Step 2: Preprocess dataset
29 df = pd.read_csv("transactions.csv")
30 print("=== Before Preprocessing ===")
31 print(df.describe(include="all"))
32 print(df.head(5))
33 print()
34
35 # Convert transaction date to datetime

```

```

35 # Convert transaction_date to datetime
36 df["transaction_date"] = pd.to_datetime(df["transaction_date"], format="%Y-%m-%d")
37
38 # Create new features
39 df["transaction_month"] = df["transaction_date"].dt.month
40 df["transaction_year"] = df["transaction_date"].dt.year
41
42 # Normalize transaction_amount using MinMaxScaler
43 scaler = MinMaxScaler()
44 df["transaction_amount_norm"] = scaler.fit_transform(df[["transaction_amount"]])
45
46 # Detect outliers using IQR and cap extreme values
47 q1 = df["transaction_amount"].quantile(0.25)
48 q3 = df["transaction_amount"].quantile(0.75)
49 iqr = q3 - q1
50 lower_bound = q1 - 1.5 * iqr
51 upper_bound = q3 + 1.5 * iqr
52
53 # Save original values for comparison
54 df["transaction_amount_capped"] = df["transaction_amount"].copy()
55 df.loc[df["transaction_amount_capped"] < lower_bound, "transaction_amount_capped"] = lower_bound
56 df.loc[df["transaction_amount_capped"] > upper_bound, "transaction_amount_capped"] = upper_bound
57
58 # Re-normalize after capping
59 df["transaction_amount_norm_capped"] = scaler.fit_transform(df[["transaction_amount_capped"]])
60
61 # Step 3: Summary and final output
62 print("=== Calculated bounds ===")
63 print(f"IQR: {iqr:.2f}, lower_bound: {lower_bound:.2f}, upper_bound: {upper_bound:.2f}")
64 print()
65
66 print("=== After Preprocessing ===")
67 print(df.describe(include="all"))
68 print(df.head(8))
69 print()

```

```

=== Before Preprocessing ===
      transaction_id transaction_date transaction_amount customer_id
count      35.000000          35          35.000000      35.000000
unique         NaN          30             NaN             NaN
top          NaN      2023-04-13             NaN             NaN
freq         NaN          2             NaN             NaN
mean     1018.000000          NaN      253.454000      8.028571
std       10.246951          NaN      328.701115      4.422441
min       1001.000000          NaN     -200.000000      1.000000
25%       1009.500000          NaN      122.390000      3.500000
1          1002      2023-04-13          241.23          7
2          1003      2023-11-27           39.12          3
3          1004      2023-05-30         -200.00         12
4          1005      2023-10-21         1500.00          1

=== Calculated bounds ===
IQR: 132.95, lower_bound: -77.03, upper_bound: 454.75

=== After Preprocessing ===
      transaction_id ... transaction_amount_norm_capped
count      35.000000 ...          35.000000
mean     1018.000000 ...          0.515752
min       1001.000000 ...          0.000000
6          1007      2023-07-07 ...          143.7300          0.4151
29
7          1008      2023-12-30 ...          28.8300          0.1990
63

[8 rows x 9 columns]

=== Outlier caps count ===
Below lower: 1
Above upper: 2
      transaction_id transaction_date ... transaction_year customer_id
0          1001      2023-04-13 ...          2023          5
1          1002      2023-04-13 ...          2023          7
2          1003      2023-11-27 ...          2023          3
3          1004      2023-05-30 ...          2023         12

```

Task 3: Environmental Sensor Data Preparation

AI Prompt Used

Generate Python code to clean sensor data by handling missing values, scaling features, encoding categorical data, and splitting dataset.

Explanation

This task focuses on preparing environmental sensor data for machine learning. Missing values in numerical columns like temperature, humidity, and air quality index are replaced using mean values. Numerical features are scaled to improve model performance. Categorical data such as sensor status is encoded into numerical format. Finally, the dataset is split into training and testing sets for model building.

Code Screenshot

```

6 # Step 1: Create sample CSV
7 np.random.seed(42)
8 n = 28
9 data = []
10 for i in range(n):
11     temp = np.random.normal(25, 5)
12     hum = np.random.normal(50, 10)
13     aqi = np.random.normal(80, 15)
14     if np.random.rand() < 0.15:
15         temp = np.nan
16     if np.random.rand() < 0.15:
17         hum = np.nan
18     if np.random.rand() < 0.15:
19         aqi = np.nan
20     status = np.random.choice(["active", "inactive", "faulty"], p=[0.6, 0.25, 0.15])
21     data.append({
22         "temperature": round(temp if not np.isnan(temp) else np.nan, 2),
23         "humidity": round(hum if not np.isnan(hum) else np.nan, 2),
24         "air_quality_index": round(aqi if not np.isnan(aqi) else np.nan, 2),
25         "sensor_status": status
26     })
27
28 df = pd.DataFrame(data)
29 df.to_csv("sensor_data.csv", index=False)
30 print("Created sensor_data.csv with sample data.\n")
31
32 # Step 2: Preprocess
33 df = pd.read_csv("sensor_data.csv")
34 print("Original NA counts:")
35 print(df.isna().sum(), "\n")
36
37 # Fill missing numeric with mean
38 for col in ["temperature", "humidity", "air_quality_index"]:
39     df[col] = df[col].fillna(df[col].mean())

```

```
C: > Users > lenovo > Desktop > 3-2 > AIAC > Code6.py > ...
35 print(df.isna().sum())
36
37 # Fill missing numeric with mean
38 for col in ["temperature", "humidity", "air_quality_index"]:
39     df[col].fillna(df[col].mean(), inplace=True)
40
41 # Scale numeric features
42 scaler = StandardScaler()
43 num_cols = ["temperature", "humidity", "air_quality_index"]
44 df[num_cols] = scaler.fit_transform(df[num_cols])
45
46 # Encode sensor_status
47 le = LabelEncoder()
48 df["sensor_status_encoded"] = le.fit_transform(df["sensor_status"])
49
50 # Train/test split 80/20
51 X = df[num_cols + ["sensor_status_encoded"]]
52 y = df["sensor_status_encoded"] # placeholder target
53 X_train, X_test, y_train, y_test = train_test_split(
54     X, y, test_size=0.2, random_state=42, stratify=df["sensor_status_encoded"]
55 )
56
57 # Step 3: Print shapes
58 print("Train shape:", X_train.shape, y_train.shape)
59 print("Test shape:", X_test.shape, y_test.shape)
60 print("\nLabel mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
61 print("\nSample processed data:")
62 print(df.head())

> & C:/Python314/python.exe c:/Users/lenovo/Desktop/3-2/AIAC/Code6.py
Created sensor_data.csv with sample data.

Original NA counts:
temperature      9
humidity          1
air_quality_index  7
sensor_status     0
dtype: int64

Train shape: (22, 4) (22,)
Test shape: (6, 4) (6,)

Label mapping: {'active': np.int64(0), 'faulty': np.int64(1), 'inactive': np.int64(2)}

Sample processed data:
   temperature  humidity  air_quality_index  sensor_status  sensor_status_encoded
0  5.057294e-01  0.065349    9.978517e-16      faulty          1
1  3.519333e-15  2.151807    7.687487e-01      active          0
2 -6.829954e-01 -0.333228    2.154342e-01      active          0
3 -2.478465e+00 -0.870336   -1.526664e+00      active          0
4  3.519333e-15  0.367928   -1.251411e+00     inactive          2
PS C:\Users\lenovo> 
```

Task 4: Smart City Traffic Data Cleaning

AI Prompt Used

Generate Python script to clean traffic dataset including standardization, missing value handling, duplicate removal, normalization, and comparison.

Explanation

In this task, traffic data collected from multiple sensors is cleaned and standardized. Road and location names are converted into a consistent format by removing extra spaces and converting to lowercase. Missing values in traffic density are filled using median values. Duplicate records are removed to avoid redundancy. Numerical features such as speed and vehicle count are normalized to ensure uniform scaling across the dataset.

Code Screenshot

```
5 # Step 1: Create sample CSV
6 np.random.seed(24)
7 rows = []
8 roads = ["Main St", "main st ", "MAIN ST", "Broadway", " broadway", "Highway 1", "highway 1 "]
9 locs = ["Downtown", "downtown ", "DOWNTOWN", "Uptown", " uptown", "Suburb", "suburb "]
10
11 for i in range(1, 36):
12     sensor_id = np.random.choice([101, 102, 103, 104, 105], p=[0.2, 0.2, 0.2, 0.2, 0.2])
13     r = np.random.choice(roads)
14     l = np.random.choice(locs)
15     density = np.random.normal(45, 20)
16     if np.random.rand() < 0.18:
17         density = np.nan
18     speed = np.random.uniform(20, 90)
19     vehicle_count = int(np.random.poisson(60))
20     rows.append({
21         "sensor_id": sensor_id,
22         "road_name": r,
23         "location": l,
24         "traffic_density": round(density, 2) if not np.isnan(density) else np.nan,
25         "speed": round(speed, 2),
26         "vehicle_count": vehicle_count,
27     })
28
29 df = pd.DataFrame(rows)
30 df.to_csv("traffic_data.csv", index=False)
31 print("Created traffic_data.csv with sample data.\n")
32
33 # Step 2: Clean and preprocess
34 df = pd.read_csv("traffic_data.csv")
35
36 print("=== Before cleaning ===")
```

```
users \lenovo \Desktop \3-2 \AIAC \Code1.py ...
median_density = df["traffic_density"].median()
df["traffic_density"] = df["traffic_density"].fillna(median_density)

# Remove duplicates on sensor_id + road_name + location + speed + vehicle_count (sensor reading)
before_dups = len(df)
df = df.drop_duplicates(subset=["sensor_id", "road_name", "location", "speed", "vehicle_count"], keep="first")
after_dups = len(df)

# Normalize speed and vehicle_count with MinMaxScaler
scaler = MinMaxScaler()
df[["speed_norm", "vehicle_count_norm"]] = scaler.fit_transform(df[["speed", "vehicle_count"]])

# Step 3: Compare quality before/after
print("=== Cleaning actions ===")
print(f"Rows before dedupe: {before_dups}, after dedupe: {after_dups}")
print(f"Filled traffic_density NaN with median={median_density:.2f}\n")

print("=== After cleaning ===")
print("Missing values:")
print(df.isna().sum())
print("\nSummary statistics:")
print(df.describe(include="all"))
print()

print("=== Sample cleaned data ===")
print(df.head(10))

print("\n=== Data quality summary ===")
print("Total rows:", len(df))
print("Total cols:", df.shape[1])
print("Columns:", list(df.columns))
```

freq	NaN	15	15	NaN	NaN	NaN	NaN	NaN
mean	103.314286	NaN	NaN	42.403143	54.298000	59.942857	0.487422	0.497403
std	1.510189	NaN	NaN	18.942857	19.228563	6.641859	0.278958	0.301903
min	101.000000	NaN	NaN	3.940000	20.700000	49.000000	0.000000	0.000000
25%	102.000000	NaN	NaN	31.395000	42.145000	54.000000	0.311113	0.227273
50%	104.000000	NaN	NaN	43.420000	55.860000	60.000000	0.510083	0.500000
75%	105.000000	NaN	NaN	54.610000	66.795000	65.000000	0.668722	0.727273
max	105.000000	NaN	NaN	77.420000	89.630000	71.000000	1.000000	1.000000

```
=== Sample cleaned data ===
sensor_id  road_name  location  traffic_density  speed  vehicle_count  speed_norm  vehicle_count_norm
0         105    main st  downtown          74.76   63.31             55    0.618163             0.272727
1         101    main st  uptown           45.93   57.39             54    0.532279             0.227273
2         103  highway 1  downtown           3.94   55.86             56    0.510083             0.318182
3         105  broadway  uptown           32.88   85.44             49    0.939214             0.000000
4         102    main st  downtown           46.23   43.52             54    0.331060             0.227273
5         102  broadway  suburb           29.62   67.90             59    0.684753             0.454545
6         104    main st  downtown           64.94   23.55             60    0.041346             0.500000
7         104  highway 1  uptown           52.81   58.04             71    0.541709             1.000000
8         101    main st  downtown           35.37   61.78             64    0.595967             0.681818
9         105    main st  uptown            9.96   66.17             52    0.659655             0.136364

=== Data quality summary ===
Total rows: 35
Total cols: 8
Columns: ['sensor_id', 'road_name', 'location', 'traffic_density', 'speed', 'vehicle_count', 'speed_norm', 'vehicle_count_norm']
PS C:\Users\lenovo>
```

Task 5: Social Media Text Cleaning and Feature Extraction

AI Prompt Used

Generate Python code to clean social media text data without using nltk, including regex cleaning, tokenization, stopwords removal, and feature extraction.

Explanation

This task involves cleaning and preprocessing textual data from social media. Duplicate and null entries are removed to ensure data quality. Text is cleaned by removing URLs, special characters, and extra spaces using regular expressions. The cleaned text is converted to lowercase and tokenized into words. Stopwords are removed to retain meaningful words. Finally, features such as word count and sentiment score are generated for further analysis.

Code Screenshot

```
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from textblob import TextBlob

# Ensure NLTK resources are available
nltk.download("punkt", quiet=True)
nltk.download("stopwords", quiet=True)
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

# Step 1: create sample CSV
rows = [
    {"post_id": 1, "text": "Love this! #sunnyday https://example.com"},
    {"post_id": 2, "text": "bad service... never again!!"},
    {"post_id": 3, "text": "OMG!!! this is the best thing EVER!! 🍕"},
    {"post_id": 4, "text": "Visit https://test.com now :)"},
    {"post_id": 5, "text": None},
    {"post_id": 6, "text": "Just meh..."},
    {"post_id": 7, "text": "JUST got a new camera, So happy."},
    {"post_id": 8, "text": "just got a new camera, so happy."}, # duplicate-like
    {"post_id": 9, "text": "Spam http://spam.org spam spam"},
    {"post_id": 10, "text": "I hate this! #fail"},
    {"post_id": 11, "text": "i Hate this! #fail"}, # duplicate-like case
    {"post_id": 12, "text": " good morning everyone "},
    {"post_id": 13, "text": "terrible interface, crashes often"},
    {"post_id": 14, "text": "Unbelievable service. 10/10"},
    {"post_id": 15, "text": "meh, could be better ;("},
    {"post_id": 16, "text": "http://skip.me This is a link then text"},
    {"post_id": 17, "text": "WOW, this product rules!!!"},
    {"post_id": 18, "text": "worst experience ever, never buying again"}
]

df.to_csv("social_media_posts.csv", index=False)
print("Created social_media_posts.csv\n")

# Step 2: clean text
df = pd.read_csv("social_media_posts.csv")
df = df.dropna(subset=["text"]).copy()
df = df.drop_duplicates(subset=["text"]).reset_index(drop=True)

def clean_text(t: str) -> str:
    t = re.sub(r"http\S+|www\S+|https\S+", "", t, flags=re.IGNORECASE) # URLs
    t = re.sub(r"[^A-Za-z0-9\s]", " ", t) # keep alpha-num and spaces
    t = re.sub(r"\s+", " ", t).strip()
    return t.lower()

df["clean_text"] = df["text"].astype(str).apply(clean_text)

tokenizer = nltk.word_tokenize
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def tokenize_and_lemmatize(text):
    tokens = tokenizer(text)
    tokens = [w for w in tokens if w not in stop_words and len(w) > 1]
    return [lemmatizer.lemmatize(w) for w in tokens]

df["tokens"] = df["clean_text"].apply(tokenize_and_lemmatize)

# Step 3: feature extraction
df["word_count"] = df["tokens"].apply(len)
df["sentiment"] = df["clean_text"].apply(lambda x: TextBlob(x).sentiment.polarity)
```

```
Cleaned dataset with features:

post_id      text      clean_text      tokens      word_count      sentiment
0      1      Love this! #sunnyday https://example.com      love this sunnyday      [love, sunnyday]      2      0.500000
1      2      bad service... never again!!      bad service never again      [bad, service, never]      3      -0.700000
2      3      OMG!!! this is the best thing EVER!! 🤩      omg this is the best thing ever      [omg, best, thing, ever]      4      1.000000
3      4      Visit https://test.com now :)      visit now      [visit]      1      0.000000
4      6      Just meh...      just meh      [meh]      1      0.000000
5      7      JUST got a new camera, So happy.      just got a new camera so happy      [got, new, camera, happy]      4      0.468182
6      8      just got a new camera, so happy.      just got a new camera so happy      [got, new, camera, happy]      4      0.468182
7      9      Spam http://spam.org spam spam      spam spam spam      [spam, spam, spam]      3      0.000000
8      10      I hate this! #fail      i hate this fail      [hate, fail]      2      -0.650000
9      11      i Hate this! #fail      i hate this fail      [hate, fail]      2      -0.650000
10     12      good morning everyone      good morning everyone      [good, morning, everyone]      3      0.700000
11     13      terrible interface, crashes often      terrible interface crashes often      [terrible, interface, crash, often]      4      -1.000000
12     14      Unbelievable service. 10/10      unbelievable service 10 10      [unbelievable, service, 10, 10]      4      0.250000
13     15      meh, could be better :(      meh could be better      [meh, could, better]      3      0.500000
14     16      http://skip.me This is a link then text      this is a link then text      [link, text]      2      0.000000
15     17      WOW, this product rules!!!      wow this product rules      [wow, product, rule]      3      0.100000
16     18      worst experience ever, never buying again      worst experience ever never buying again      [worst, experience, ever, never, buying]      5      -1.000000
17     19      happy happy happy :)      happy happy happy      [happy, happy, happy]      3      0.800000
18     20      Bad, bad, bad... yet okay sometimes.      bad bad bad yet okay sometimes      [bad, bad, bad, yet, okay, sometimes]      6      -0.400000

Data summary:
word_count      sentiment
count      19.000000      19.000000
mean      3.105263      -0.005981
std      1.286457      0.607143
min      1.000000      -1.000000
25%      2.000000      -0.525000
50%      3.000000      0.000000
75%      4.000000      0.484091
max      6.000000      1.000000

PS C:\Users\lenovo>
```

Conclusion

In this lab, various datasets including academic, financial, environmental, traffic, and social media data were cleaned and preprocessed using AI-assisted approaches. Techniques such as handling missing values, removing duplicates, encoding categorical variables, normalization, and feature extraction were applied effectively. These preprocessing steps significantly improve data quality and ensure better performance of machine learning models. Additionally, the use of AI-generated scripts helps in automating repetitive tasks, saving time, and reducing human errors.